

JAVA

Abstração e Interfaces

Classes Abstratas • Interfaces • JDK 8/9 • Polimorfismo

Apostila completa com explicações detalhadas e exemplos práticos para consolidar abstração em Java.

CONTEÚDO DA APOSTILA

- 01** Introdução: Generalizar e Abstrair
- 02** Classes Abstratas
- 03** Interfaces — Fundamentos
- 04** Interfaces — Recursos Avançados
- 05** Novidades do JDK 8 e JDK 9
- 06** Interface vs. Classe Abstrata
- 07** Boas Práticas e Erros Comuns

Sumário

- 01 Introdução: Generalizar e Abstrair**
O raciocínio por trás de toda hierarquia de classes

- 02 Classes Abstratas**
Estado compartilhado, construtores e métodos concretos

- 03 Interfaces — Fundamentos**
O contrato de comportamento e as constantes implícitas

- 04 Interfaces — Recursos Avançados**
Herança entre interfaces e coding to an interface

- 05 Novidades do JDK 8 e JDK 9**
Métodos default, static e private

- 06 Interface vs. Classe Abstrata**
Comparação direta e critérios de escolha

- 07 Boas Práticas e Erros Comuns**
Padrões de projeto e armadilhas para evitar

01 Introdução: Generalizar e Abstrair

Por que agrupar conceitos comuns é a base do bom design orientado a objetos

Toda linguagem orientada a objetos precisa resolver o mesmo problema básico: como representar coisas parecidas sem repetir código para cada uma delas. Java resolve isso com dois conceitos que caminham juntos — generalização e abstração. Entender a diferença entre eles ajuda a enxergar por que classes abstratas e interfaces existem e quando cada uma faz sentido.

Generalização: encontrando o que é comum

Generalizar é olhar para vários objetos diferentes e perceber o que eles têm em comum. Um violão, um piano e uma bateria são instrumentos muito diferentes entre si, mas todos produzem som, podem ser afinados (de alguma forma) e pertencem à categoria mais ampla de 'instrumento musical'. Esse processo de subir um nível de generalidade é exatamente o que fazemos ao desenhar uma hierarquia de classes: no topo fica o conceito mais genérico, e conforme descemos, os tipos ficam mais específicos e concretos.

Abstração: descrevendo um grupo, não um exemplar

A abstração é a consequência natural da generalização. Quando criamos o conceito 'Instrumento Musical', não estamos pensando em nenhum instrumento específico — estamos descrevendo características e comportamentos que qualquer instrumento deveria ter. Não existe, no mundo real, um objeto que seja apenas um 'Instrumento Musical' genérico; existe sempre uma guitarra, um violino, um tambor. Um bom teste mental: se você não consegue apontar para um exemplo único e definitivo daquele conceito, provavelmente ele é abstrato.

Em termos de código, a regra é simples: se uma classe pode ser instanciada diretamente com `new`, ela é concreta. Se o próprio compilador impede essa instanciação, ela é abstrata. Java oferece duas ferramentas para modelar esse tipo de conceito — a classe abstrata e a interface — e o restante desta apostila mostra como cada uma funciona.

O método abstrato como promessa de comportamento

Um método abstrato tem assinatura — nome, parâmetros e tipo de retorno — mas não tem corpo. Ele declara o que um objeto deve saber fazer, sem dizer como. É uma promessa: toda classe concreta que descende dele se compromete a fornecer sua própria implementação antes de poder ser instanciada.

```
abstract class InstrumentoMusical {  
  
    // sem corpo — apenas a assinatura do método:  
    abstract void tocar();  
    abstract void afinar();  
}
```

Exemplo — dois métodos abstratos dentro de uma classe abstrata

02 Classes Abstratas

Declaração, estado compartilhado, construtores e hierarquias

Como declarar e o que ela pode conter

Para tornar uma classe abstrata, basta acrescentar a palavra-chave `abstract` antes de `class`. A partir daí, o compilador bloqueia qualquer tentativa de criar uma instância direta dessa classe — ela só serve como base para outras classes, que devem estendê-la e preencher o que falta.

A grande diferença de uma classe abstrata em relação a uma interface é que ela se comporta como qualquer outra classe em quase tudo: pode ter construtor (chamado pelas subclasses com `super()`), pode guardar estado por meio de campos de instância, e pode misturar métodos abstratos com métodos já prontos, que todas as subclasses herdam sem precisar reescrever.

```
// não é possível fazer 'new InstrumentoMusical(...)'
abstract class InstrumentoMusical {
    private String fabricante;
    private int anoFabricacao;

    public InstrumentoMusical(String fabricante, int ano) {
        this.fabricante = fabricante;
        this.anoFabricacao = ano;
    }

    // obrigatório nas subclasses concretas:
    public abstract void tocar();
    public abstract void afinar();

    // pronto e compartilhado por todas as subclasses:
    public String getFabricante() { return fabricante; }
}
```

Exemplo — construtor, estado e mistura de métodos abstratos e concretos

Hierarquias com mais de um nível

Uma subclasse de uma classe abstrata não é obrigada a implementar todos os métodos herdados — ela pode continuar abstrata e empurrar essa responsabilidade adiante. A regra do compilador é clara: só quando a cadeia de herança chega a uma classe que não é mais declarada como `abstract` é que todos os métodos abstratos pendentes precisam finalmente ganhar implementação.

```
abstract class InstrumentoMusical {
    abstract void tocar();
    abstract void afinar();
}

// ainda abstrata — adiciona um novo método abstrato:
abstract class InstrumentoDeCorda extends InstrumentoMusical {
    abstract void trocarCorda();
}

// concreta — implementa TUDO que ficou pendente:
class Violao extends InstrumentoDeCorda {
    @Override public void tocar() { System.out.println("Dedilhando o violão"); }
    @Override public void afinar() { System.out.println("Ajustando as cravelhas"); }
    @Override public void trocarCorda() { System.out.println("Trocando a corda solta"); }
}
```

Exemplo — cadeia de herança com dois níveis de abstração

QUANDO USAR CLASSE ABSTRATA

Prefira classe abstrata quando as subclasses compartilham estado real (campos como fabricante, ano, identificador) e também comportamento concreto em comum. Ela funciona como um esqueleto pronto: as subclasses só completam as partes que realmente mudam de um tipo para outro.

03 Interfaces — Fundamentos

O contrato de comportamento, implements e constantes

Um contrato, não uma classe

Uma interface não é uma classe — é um tipo próprio que define um contrato puro de comportamento. Ela descreve o que uma classe deve ser capaz de fazer, sem revelar nenhum detalhe de como isso é feito nem qual estado interno é necessário. Se uma classe abstrata responde 'o que você é' (uma relação de herança, is-a), uma interface responde 'o que você sabe fazer' (uma capacidade, can-do).

A vantagem mais evidente das interfaces é a possibilidade de implementar várias ao mesmo tempo — uma classe só pode estender uma única superclasse, mas pode implementar quantas interfaces forem necessárias. Isso permite tratar de forma uniforme objetos que não têm nenhum parentesco real entre si, desde que compartilhem o mesmo comportamento.

```
public interface Afinavel {
    void afinar(); // implicitamente public abstract
}

public interface Amplificavel {
    void conectarAmplificador();
}

// uma classe pode implementar mais de uma interface:
class GuitarraEletrica extends InstrumentoDeCorda
implements Afinavel, Amplificavel {
    @Override public void tocar() { System.out.println("Solando na guitarra"); }
    @Override public void afinar() { System.out.println("Afinando por referência
    eletrônica"); }
    @Override public void trocarCorda() { System.out.println("Trocando corda de aço"); }
    @Override public void conectarAmplificador() { System.out.println("Plugando no
    amplificador"); }
}
```

Exemplo — uma classe estendendo uma classe abstrata e implementando duas interfaces

Modificadores implícitos e constantes

Interfaces têm regras próprias sobre modificadores de acesso. Qualquer método declarado sem corpo dentro de uma interface é, por padrão, `public abstract` — mesmo que você não escreva esses modificadores. Isso contrasta com classes comuns, onde a ausência de modificador significa acesso de pacote (package-private); em interfaces, a ausência sempre significa público.

Campos declarados dentro de uma interface também seguem uma regra fixa: são sempre `public static final`, ou seja, constantes. Por convenção, elas são escritas em letras maiúsculas separadas por underscore, e acessadas diretamente pelo nome da interface.

```
public interface Amplificavel {
    int VOLUME_MAXIMO = 10; // public static final por padrão
    void conectarAmplificador();
}

// acesso via nome da interface, sem instância:
System.out.println(Amplificavel.VOLUME_MAXIMO); // 10
```

Exemplo — constante implícita em uma interface

04 Interfaces — Recursos Avançados

Herança entre interfaces, polimorfismo e coding to an interface

Tratando tipos diferentes de forma uniforme

A maior força de uma interface aparece quando objetos sem nenhum parentesco de herança compartilham o mesmo comportamento. Uma guitarra elétrica e um teclado sintetizador não têm nada

em comum na árvore de herança, mas ambos podem ser 'amplificáveis'. Isso permite guardar referências completamente diferentes sob o mesmo tipo de interface e tratá-las de forma idêntica em um laço, sem saber (nem precisar saber) o tipo concreto de cada uma.

```
Amplificavel[] equipamentos = { new GuitarraEletrica(), new TecladoSintetizador() };  
for (Amplificavel eq : equipamentos) {  
    eq.conectarAmplificador(); // cada um executa sua própria versão  
}
```

Exemplo — polimorfismo por interface em um array heterogêneo

Interfaces também herdam de interfaces

Assim como classes herdam de outras classes, uma interface pode estender outra interface — e, diferentemente das classes, pode estender várias ao mesmo tempo. A interface filha herda todos os métodos abstratos das interfaces pai. Um detalhe importante de sintaxe: interfaces sempre usam `extends` entre si, nunca `implements`.

```
public interface Multimidia extends Afinavel, Amplificavel {  
    void gravar();  
}  
  
// INVÁLIDO — interface não usa 'implements':  
// public interface Multimidia implements Afinavel { } // erro de compilação
```

Exemplo — uma interface estendendo duas outras ao mesmo tempo

Coding to an interface

Programar voltado para a interface significa declarar variáveis, parâmetros e retornos de método usando o tipo da interface (ou de uma classe abstrata), em vez do tipo concreto que está sendo criado. Essa prática desacopla o código de uma implementação específica — trocar a implementação passa a exigir mudança em um único ponto, sem afetar o resto do sistema.

```
// acoplado à implementação concreta:  
ArrayList<String> playlist = new ArrayList<>();  
  
// programando para a interface — melhor prática:  
List<String> playlist = new ArrayList<>();  
  
// trocar a implementação exige mudar só esta linha:  
List<String> playlist = new LinkedList<>();
```

Exemplo — coding to an interface aplicado a uma coleção

05 Novidades do JDK 8 e JDK 9

Métodos default, static e private dentro de interfaces

O problema que o método default resolveu

Antes do JDK 8, uma interface só podia declarar métodos abstratos. Isso criava um problema sério de evolução: assim que uma nova versão da interface ganhava um método a mais, toda classe que já a implementava parava de compilar, porque não fornecia aquele novo método. Em um sistema com dezenas de implementações espalhadas, isso significava reescrever todas elas de uma vez.

O método default resolveu isso permitindo que uma interface forneça uma implementação padrão para um método. Classes que já existiam continuam funcionando sem nenhuma alteração, herdando automaticamente esse comportamento — a mudança se torna retrocompatível.

```
public interface Amplificavel {
    void conectarAmplificador();

    // método concreto dentro da interface – pode ser sobrescrito:
    default void verificarCabo() {
        System.out.println("Cabo P10 verificado – conexão padrão OK");
    }
}

// classes existentes não precisam implementar verificarCabo()
```

Exemplo — método default garantindo retrocompatibilidade

Três formas de lidar com um método default

Ao implementar uma interface que traz métodos default, uma classe tem três caminhos possíveis para cada um deles: não sobrescrever e simplesmente herdar o comportamento padrão; sobrescrever por completo com uma lógica própria; ou sobrescrever mantendo parte do comportamento original, chamando-o explicitamente com a sintaxe `NomeDaInterface.super.metodo()`.

```
// opção 1: não sobrescrever – herda verificarCabo() como está

// opção 2: sobrescrever por completo
@Override
public void verificarCabo() {
    System.out.println("Cabo balanceado XLR verificado");
}

// opção 3: reaproveitar a implementação padrão e complementar
@Override
public void verificarCabo() {
    Amplificavel.super.verificarCabo();
    System.out.println("+ checagem extra de ruído");
}
```

Exemplo — as três formas de tratar um método default

Métodos static (JDK 8) e private (JDK 9)

O JDK 8 também passou a permitir métodos `static` dentro de interfaces — utilitários ligados ao contrato da interface, mas que não pertencem a nenhuma instância específica. Eles são chamados

diretamente pelo nome da interface, e costumam ser úteis como métodos de fábrica ou utilidades gerais relacionadas ao tema da interface.

Já o JDK 9 introduziu métodos `private` em interfaces — tanto de instância quanto estáticos. Eles existem para evitar duplicação de código entre métodos `default`: a lógica repetida é extraída para um método privado que só outros métodos da própria interface conseguem chamar.

```
public interface Amplificavel {  
  
    // método static – utilitário da própria interface:  
    static String descreverPadrao(Amplificavel a) {  
        return "Equipamento: " + a.getClass().getSimpleName();  
    }  
  
    // método private (JDK 9) – reuso interno entre defaults:  
    private void log(String msg) {  
        System.out.println("[amplificacao] " + msg);  
    }  
  
    default void verificarCabo() {  
        log("cabo verificado");  
    }  
}
```

Exemplo — método static e método private trabalhando juntos

06 Interface vs. Classe Abstrata

Comparação direta e critérios objetivos para escolher entre as duas

Característica	Classe Abstrata	Interface
Campos de instância	Sim, com estado próprio	Só constantes (<code>public static final</code>)
Construtor	Sim, chamado via <code>super()</code>	Não existe
Modificadores de acesso	Qualquer um	Sempre <code>public</code>
Herança múltipla	Estende só uma classe	Implementa várias interfaces
Palavra-chave na classe	<code>extends</code>	<code>implements</code>
Métodos concretos	Sim, livremente	Só <code>default</code> , <code>static</code> e <code>private</code>
Relação semântica	is-a (é um)	can-do (sabe fazer)

Critérios práticos de escolha

Opte por classe abstrata quando os tipos envolvidos são claramente parentes, compartilham estado real (como fabricante, identificador, configuração) e também compartilham código concreto que não

muda de subclasse para subclasse. Ela é a escolha certa quando o relacionamento é genuinamente de herança — 'uma Guitarra Elétrica é um Instrumento de Corda'.

Opte por interface quando tipos sem parentesco algum precisam apresentar o mesmo comportamento, quando o objetivo é apenas definir um contrato sem se preocupar com estado interno, ou quando uma mesma classe precisa assumir vários papéis simultâneos. Desde o JDK 8, com métodos default, interfaces ficaram ainda mais flexíveis e reduziram boa parte das situações em que só a classe abstrata resolvia.

07 Boas Práticas e Erros Comuns

Padrões de projeto que usam abstração e armadilhas para evitar

Erros frequentes

```
// tentar instanciar uma classe abstrata ou interface diretamente:
InstrumentoMusical i = new InstrumentoMusical(); // erro – é abstrata!
Amplificavel a = new Amplificavel(); // erro – é interface!

// esquecer de implementar um dos métodos abstratos herdados:
class Bateria implements Amplificavel {
    @Override public void conectarAmplificador() { /* ok */ }
    // se Amplificavel tivesse outro método abstrato e ele não
    // fosse implementado aqui, o código não compilaria
}

// usar 'implements' entre interfaces em vez de 'extends':
// public interface X implements Y { } // errado, deveria ser extends
```

Exemplo — erros de compilação comuns envolvendo abstração

Padrões de projeto associados

Padrão	Base	Ideia central
Template Method	Classe Abstrata	Define o esqueleto de um algoritmo; subclasses preenchem as etapas
Abstract Factory	Classe Abstrata	Cria famílias de objetos relacionados a partir de uma hierarquia abstrata
Strategy	Interface	Permite trocar um algoritmo por outro em tempo de execução
Observer	Interface	Notifica múltiplos ouvintes quando um evento acontece
Decorator	Interface	Envolve uma implementação para acrescentar comportamento extra

BOA PRÁTICA

Nomeie interfaces como comportamentos, não como substantivos genéricos — `Amplificavel`, `Comparable`, `Serializable`. O nome já deixa claro o que qualquer classe que a implemente é capaz de fazer.

Resumo Final

Abstração	Representa um grupo de comportamentos, não um exemplar concreto do mundo real.
Classe abstrata	Fornece estado e código concreto compartilhado entre tipos parecidos.
Interface	Define um contrato de comportamento para tipos que não precisam ter parentesco.
default / static / private	Desde o JDK 8/9, interfaces podem ter métodos com implementação própria.
Coding to an interface	Programe contra o tipo mais genérico possível para reduzir acoplamento.